CS781 Capstone Project: In-House Legal Intelligence System

Sunny Kumar Pandit
231055
sunnyp23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)
Code: github

Abstract

A law firm that cannot send case files to external APIs needs a self-hosted legal language system. We built a seven-phase pipeline around one compact foundation model per architecture family: a 145M-parameter decoder-only LLaMA-style transformer pre-trained from scratch on 400M tokens of Indian legal text (Supreme Court 40%, Acts 30%, High Courts 20%, Tribunals 10%) with a custom 48k BPE tokenizer. We compared it with a local/global hybrid, a gated linear-attention hybrid, and an encoder–decoder. Every later task (NER, rhetorical roles, judgment prediction, statute identification, summarization, case retrieval) starts from the Phase-1 weights and is adapted with full fine-tuning or parameter-efficient methods (LoRA, LoRMA, adapters, prefix/soft prompts, distillation). Architecture 1 achieved the highest leaderboard score in all seven phases. In the Phase 2 and Phase 3 comparisons, parameter-efficient methods generally matched or outperformed full fine-tuning, although the best adaptation method varied between architectures. However, our controlled comparisons show where those conclusions are confounded.

1 Competition Result

Codabench username: s_231055 **Roll number:** 231055

Table 1: Best valid official score per phase. BS-F1 = BERTScore F1.

Phase	Task	Best system	Primary metric (value)	Rank	Valid subm.
1	Legal LM pretraining	Arch 1, modified	BS-F1 (0.8449)	8	23
2	Legal NER	Arch 1 + LoRMA	strict macro-F1 (0.5610)	39	23
3	Rhetorical roles	Arch 1 + LoRA	macro-F1 (0.5102)	51	18
4	Judgment prediction	Arch 1 + LoRMA	macro-F1 (0.7492)	49	7
5	Statute identification	Arch 1 + LoRMA	macro-F1 (0.3559)	86	8
6	Summarization	Arch 1 + LoRA	BS-F1 (0.8437)	50	4
7	Prior case retrieval	Arch 1 + LoRA	micro-F1@7 (0.2304)	119	4

Total valid submissions: 87. **Per-phase minimum of four distinct submissions:** Yes; we ran four architectures per phase, so at least four distinct systems exist for each phase.

2 Introduction and Problem Description

A firm handling privileged case material cannot rely on hosted language models, so the capstone asks for a small, self-hosted system that can read, label and generate legal text. We therefore built

one pipeline of seven tasks on top of a single pre-trained backbone per architecture family, trained under the constraints of the course: a Phase-1 model of 50M–300M parameters, no pre-trained language models, no external legal corpora, and Kaggle T4 compute.

The tasks are as follows. *Phase 1* is next-token pre-training: given a 512-word prompt the model writes a 128-word continuation, scored by BERTScore F1 against the hidden reference [1]. *Phase 2* (L-NER) asks for typed character spans over 12 entity types, scored by strict span-level macro-F1, where start, end and label must all match. *Phase 3* assigns one of 13 rhetorical roles to each sentence in Corporate Law and Information Technology judgments (macro-F1). *Phase 4* predicts a binary judgment outcome from a full Supreme Court case (macro-F1). *Phase 5* is multi-label identification of the relevant statutes among 100 candidates (macro-F1). *Phase 6* writes an abstractive summary of a long judgment (BERTScore F1, ROUGE [2] as a secondary check). *Phase 7* returns the top seven prior cases from a candidate pool for each query case (micro-F1@7).

Four research questions drove our experiments. (i) Which architecture family gives the best compact legal language model: a LLaMA-style decoder, a local/global attention hybrid, a gated linear-attention hybrid, or an encoder–decoder? (ii) Do the architectural modifications beyond each baseline help, once parameter count is held roughly fixed? (iii) Does the Phase-1 backbone transfer, and which adaptation method (full fine-tuning, LoRA [3], LoRMA [4], adapters [5], prefix/soft prompts [6, 7], BitFit [8], distillation [9]) works best on small legal datasets? (iv) How should long documents be handled (chunking, hierarchical pooling, head/tail selection)? We built all seven phases end to end. We did not train a tokenizer variant ablation, use external corpora, or pipeline phases into one another; each task descends separately from the Phase-1 checkpoint.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

All data come from the course dataset on Hugging Face (`Exploration-Lab/CS-781-Capstone`). Phase 1 uses four corpora (Supreme Court, Acts, High Courts, Tribunals) for pre-training and 1,000 held-out test prompts. Table 2 lists the downstream splits that we used. For Phases 2, 3 and 6 we carved a local validation set from the official training data (8 of 70 NER documents; the official 5+5 development documents for rhetorical roles; 351 of 7,030 summarization documents). Phases 4, 5 and 7 supply official development splits.

Table 2: Downstream data used (train / dev / test).

Phase	Unit	Train / dev / test	Notable statistics
2	documents	70 (8 held out) / – / 35	8,393 entities; mean 6,800 tokens/doc
3	sentences	16,845 / 2,142 / 2,197	13 roles; CL 36 and IT 37 tokens/sentence
4	cases	37,387 / 1,988 / 1,517	59%/41% class 0/1; mean 3,269 tokens
5	documents	42,750 / 10,181 / 13,019	100 statutes; 3.7 labels/doc
6	documents	6,679 / 351 / 100	references average 780 words
7	queries	827 / 118 / 237	4,320 / 1,023 / 1,727 candidates

3.2 Corpus statistics that informed design

Phase 1. We drew token budgets by source so that the training mixture was controlled rather than proportional to file size (the High Court and Tribunal files are 10.7 GB and 6.5 GB, against 1.3 GB for the Supreme Court). Arch 1, 3 and 4 used 40/30/20/10 percent for Supreme Court/Acts/High Courts/Tribunals; the Arch 2 run used 50/25/15/10. This over-weights the Supreme Court and

Acts (70%). At earlier stages the model was trained only on Supreme Court Data and it was observed that it performed at par to the split. The split was only introduced to ingest variability. A 5% random share of documents went to validation.

Phase 2. The entity distribution is skewed: DATE (1,575), STAT (1,260), PREC (1,183), COURT (1,013) and AUTH (1,003) dominate, while WIT (187), R.COUNSEL (184) and A.COUNSEL (195) are rare. About 90.8% of tokens carry the `O` tag, and there are no nested or overlapping gold spans. Documents average 6,800 tokens, so they cannot be read in one pass. Many windows consist of boilerplate only.

Phase 3. Roles are very imbalanced: Ratio of the Decision (25.5%) and Fact (24.3%) account for half of all sentences, whereas Issue has 107 training sentences (0.6%) and Precedent Overruled just 3. Corporate Law documents are longer (up to 1,816 sentences) than IT ones (up to 442).

Phase 4. Training is 59% label 0, but dev is exactly balanced (994/994), so a threshold tuned on training prior would be mis-set. The median case has 2,443 tokens but the 90th percentile has 7,333, so truncation matters. In the logs the final sentences of label-0 cases contain explicit dispositions ("we set aside ... dismiss"), which suggests that the outcome sits at the end of the judgment. We tested this directly (Section 7).

Phase 5. All 100 statutes have at least 50 training documents, so there is no extreme long tail. Dev documents have 1 label (1,544), 2–3 labels (3,722) or $\geq$4 labels (4,915). Only 15.9% of documents exceed eight 512-token windows. **Phase 6.** References average 780 words, but we feed 1,024 tokens and generate at most 256 tokens, so summaries are necessarily short. The lead baseline (first words of the document) reaches ROUGE-L 0.266. **Phase 7.** Queries average about 1,950 tokens, with 8.2 gold cases per query on dev and 6.4 on train; since the metric is capped at seven results per query, high gold counts limit recall.

3.3 Preprocessing

Tokenization. The tokenizer is a byte-level BPE with a word-end marker [10] and 48,000 entries (all ids unique). It splits on whitespace, so punctuation stays attached to words and legal citations such as "Cr.P.C." and "Section 302" are only segmented by learned merges. For NER we needed exact character offsets, so we first cut text into words and single punctuation marks, BPE-encode each piece, and recover each token's character span from its byte length (the `</w>` marker has zero width). With whitespace-only cutting only 66.0% of gold spans were recoverable at exact boundaries; with word-plus-punctuation cutting 99.4% were. We verified the whole pipeline with an oracle test: gold spans $\rightarrow$ tags $\rightarrow$ perfect log-probabilities $\rightarrow$ decoder gives strict macro-F1 of 0.994, the ceiling imposed by the tokenization (50 spans with inexact boundaries and 10 with no token). We also confirmed that gold end offsets are exclusive (8,393/8,393 matches).

Sentences and long documents. Phase 3 uses the provided sentence lists, truncated at 96 tokens, and packs sentences into windows of 768 core tokens plus 256 tokens of left context. Phase 4 cuts cases into overlapping 1,024-token chunks (stride 896), keeping the head and tail of very long cases (maximum 12,288 tokens, up to 14 chunks). Phase 5 uses 512-token windows (stride 384, at most eight evenly spaced). Phase 6 keeps the first 60% and last 40% of the 1,024-token input window. Phase 7 encodes queries and candidates with the same truncation policy: at most four 512-token chunks, with the last chunk shifted back to stay full length; every split uses its own candidate pool.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

We built four architecture families in a shared code base (hand-written, no pre-trained weights), each with a *baseline* close to a published design and a *modified* version. All use pre-norm RMSNorm [11], SwiGLU feed-forward blocks [12] ($d_{ff} = 2048$), $d_{model} = 768$, 12 heads, and a 2,048-token context. Parameter counts are computed in code, with an assertion that they lie within 50M–300M.

Arch 1 (decoder). The baseline is LLaMA-style [13]: 12 layers, RoPE [14], multi-head attention, untied embeddings (158.7M parameters). The modified model adds grouped-query attention with 4 KV heads [15], QK-norm [16], tied embeddings, value residuals [17], two multi-token-prediction heads [18] with weight 0.3, and a source embedding (Act/High Court/Supreme Court/Tribunal); the parameters saved by tying and GQA are re-spent on depth, giving 17 layers and 145.0M parameters (108.2M non-embedding). **Arch 2 (local/global).** The baseline uses Mistral-style sliding-window attention [19] of 512 tokens in every layer. The modified model grows the window from 128 to 512 with depth and makes every fourth layer full attention without positional encoding (NoPE [20]), with GQA, QK-norm and tied embeddings (145.0M). **Arch 3 (linear attention).** The baseline is RetNet-style retention with fixed per-head decay [21]. The modified model is GLA-style [22]: input-dependent decay gates initialised from fast to slow forgetting, a short causal convolution of width 4 on q, k, v, L2-normalised q, k, an output gate, and a softmax NoPE layer every fourth layer, 16 layers in total. We implemented chunkwise-parallel training (chunk 64) and a recurrent state for generation. **Arch 4 (encoder–decoder).** The first 768 tokens of each 2,048-token block go to a bidirectional encoder, and the decoder models the remaining 1,280 while cross-attending. The baseline has 6+6 layers (172.8M); the modified model has 7 encoder and 8 decoder layers, cross-attention every second decoder layer, 2× compressed encoder memory, GQA, QK-norm, tied embeddings and MTP (189.4M).

Training. All models use next-token cross-entropy (plus the MTP terms for modified variants), AdamW [23] with $\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping 1.0, warmup then cosine decay to 0.1× the peak rate, fp16 with loss scaling, and gradient checkpointing, on two T4 GPUs. Peak rates were 6×10^{-4} (Arch 1), 5×10^{-4} (Arch 2) and 4×10^{-4} (Arch 3, 4). One epoch over 400M tokens was the budget. Sessions are capped at 12 hours, so every run saves a full checkpoint (model, optimizer, scheduler clock) every 250 steps and resumes. We implemented a KV cache and a one-token-per-step recurrent state so that generation does not re-read the prefix.

Decoding. Continuations are limited to 128 words and trimmed at a word boundary. The notebooks differ: Arch 3 samples directly (T=0.1, top-k 50, top-p 0.95, no repetition penalty); Arch 1, 2 and 4 contain a minimum-Bayes-risk (MBR) selector [24] that draws 16 samples (T=0.4, top-k 20, top-p 0.88, repetition penalty 1.08) and keeps the sample with the highest average agreement with the others, 0.9×BERTScore + 0.1×ROUGE-L.

4.2 Transfer protocol (Phases 2–7)

Every task model is initialised from the Phase-1 weights of one architecture, with the tokenizer unchanged, and gets a task head that is trained from scratch (Figure 1). Decoders are causal, so heads that need right context add a small bidirectional GRU [25] (Phases 2, 3) or pool windows with attention (Phases 4, 5); for the encoder–decoder only the encoder is used in Phases 3–5 and 7. The backbone is adapted with one of: full fine-tuning; LoRA ($W' = W_0 + \frac{\alpha}{r}BA$, rank 32 in Phases 2–3); LoRMA, a multiplicative update $W' = (I + \frac{\alpha}{r}BA)W_0$ computed as $z = W_0x$, $y = z + \frac{\alpha}{r}B(Az)$; Houlsby bottleneck adapters; prefix tuning (32 key/value vectors per layer), soft

prompts (16 virtual tokens), BitFit; QLoRA (4-bit NF4 backbone [26]); or knowledge distillation from an 8-layer teacher to a 4-layer student. All of these were written by hand for the model's `nn.Linear` layers, with a test that each adapted model starts exactly at the Phase-1 function. The compulsory LoRMA method is used for every architecture.

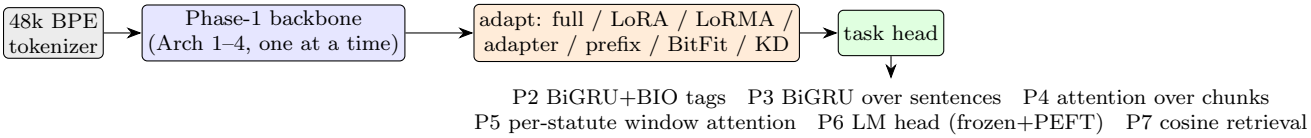


Figure 1: One backbone per architecture, seven separately fine-tuned descendants.

4.3 Phase 2 — Legal NER

We use token classification and reconstruct offsets (Section 3.3), because span extraction would not be a good fit for entities that cross many BPE tokens. Each of the 12 labels gets B- and I- tags plus a shared O (25 tags). Documents are cut into overlapping windows (2,048 tokens, stride 1,024; 1,028/512 for Arch 2), a BiGRU over the backbone states supplies right context, and the loss is class-weighted cross-entropy with weight $(n_O/n_{label})^p$ ($p \in [0.3, 0.6]$, capped at 10) while entity-free windows are down-sampled to 50–60% per epoch. At inference we compare greedy decoding with Viterbi decoding constrained to BIO rules (no O→I-x or B-x→I-y), and we drop spans whose mean token probability is below τ (searched over 0–0.7). Overlaps cannot arise because the BIO path is single-label. Arch 4 used knowledge distillation (8-layer teacher, student keeps layers 0, 2, 4, 6).

4.4 Phase 3 — Rhetorical roles

A window of core sentences plus left context goes through the backbone; each sentence becomes one vector (mean of token states, state at the last token, and a domain embedding), a 2-layer BiGRU over sentence vectors adds document context, and a linear layer predicts the role. CL and IT are trained jointly with a learned domain embedding. The loss uses label smoothing 0.05 and class weights $(N/(C\,n_c))^{0.5}$. A sentence-only variant is available but we did not run the ablation. We compared full fine-tuning, LoRA, LoRMA, prefix tuning (Arch 1, 2), QLoRA (Arch 3, implemented) and distillation for Arch 4.

4.5 Phase 4 — Court judgment prediction

A case is a sequence of overlapping 1,024-token chunks; each chunk vector is the mean and last-token state of the backbone, and a hierarchical head adds an embedding of the chunk's relative position, pools chunks with learned attention (alternatives: mean, GRU, last) and applies an MLP. Training uses 8 cases per step for 3,000 steps with class-weighted loss, and the decision threshold is tuned on dev. A TF-IDF logistic baseline serves as a sanity check; a deliberate "overfit eight documents" test checks the pipeline.

4.6 Phase 5 — Statute identification

Each 512-token window is mean-pooled, then each of the 100 statutes has its own attention over the windows (label-wise attention), followed by a linear layer and sigmoid. The loss is binary cross-entropy with pos_weight = $\text{clip}(\sqrt{n_{neg}/n_{pos}}, 1, 10)$; one global threshold is tuned on dev (per-label thresholds changed micro-F1 by only 0.002). We compared soft prompts, adapters, LoRA and LoRMA.

4.7 Phase 6 — Abstractive summarization

We use decoder-only continuation from the Phase-1 LM head "document ⟨sep⟩ summary" (encoder–decoder for Arch 4, encoder reading 768 tokens), with the base model frozen and only 0.01–0.8% of parameters trained (LoRA for Arch 1, LoRMA for Arch 2, adapters for Arch 3, soft prompt for Arch 4). Documents longer than 1,024 tokens keep their first 60% and last 40%. Decoding is greedy with repetition penalty 1.15 and no repeated 3-grams, up to 256 tokens. We did not use any post-training or alignment method.

4.8 Phase 7 — Prior case retrieval

A bi-encoder: one shared backbone encodes a case as up to four 512-token chunks, mean-pools each chunk and averages chunk vectors to a case vector, and cosine similarity ranks candidates. Training is contrastive (InfoNCE [27]) with in-batch negatives plus one mined hard negative per query, drawn from the 30 non-relevant training candidates that the frozen model finds most similar. A zero-shot frozen model is the baseline, and we ablate pooling, number of chunks, similarity, loss and negatives. Failed approaches are reported in Section 5.3.

5 Experiments and Implementation Details

5.1 Compute and training protocol

All runs used Kaggle sessions with two T4 GPUs (the Arch 3 statute notebook used one), fp16 autocast with gradient scaling, AdamW, linear warmup with cosine decay and gradient clipping 1.0. Phase-1 pre-training took 141–771 minutes per run (Table 4); downstream runs took 10–160 minutes (e.g. Phase 2: 15–30 min, Phase 4: 76–156 min, Phase 7: 10–27 min). Pre-training survived session limits through resumable checkpoints; downstream notebooks stop early on validation F1 (patience 3 in Phase 3; best-epoch selection in Phase 2; best quick-dev step in Phase 4). Table 3 gives hyperparameters of the best system in each phase.

Table 3: Hyperparameters of the best system in each phase (all Arch 1).

	P1	P2	P3	P4	P5	P6	P7
Adaptation	–	LoRMA	LoRA	LoRMA	LoRMA	LoRA	LoRA
Rank / α	–	32/64	32/64	32/64	8/16	8/16	8/16
LoRA targets	–	7 lin.	7 lin.	7 lin.	q,k,v,o	–	–
Parameters (M)	145.0	–	–	152.7	–	151.5	–
Trainable	all	–	–	5.8%	0.56M	0.38%	–
Context	2048	2048	768+256	1024	512	1024	4×512
Backbone LR	6×10^{-4}	$3\times10^{-4\dagger}$	$3\times10^{-4\dagger}$	1×10^{-4}	3×10^{-4}	3×10^{-4}	5×10^{-4}
Head LR	–	10^{-3}	10^{-3}	5×10^{-4}	10^{-3}	–	–
Batch×accum.	8×16	8×2 win.	4×4 win.	2×4 docs	4×2 docs	2×8	4×4 q.
Epochs / steps	1525 st.	≤25 ep.	≤10 ep.	3000 st.	3 ep.	2 ep.	2 ep.
Dropout	0	0.05	0.05	0.05 / 0.1	head 0.1	–	–
Decoding / thr.	MBR-16	Viterbi, τ	argmax	tuned thr.	global thr.	greedy+rep.	cosine top-7

5.2 Local evaluation

Because only a few labelled documents exist in some phases, our local validation sets are small: 8 NER documents, 5+5 role documents, 351 summaries, and 118 retrieval queries. Local and leaderboard scores diverged accordingly. Arch 1 NER scored 0.606 locally (full fine-tuning, Viterbi, $\tau = 0.7$) against 0.539 on the leaderboard; Arch 1 role prediction 0.550 against 0.491; Phase 7 Arch 3

0.242 against 0.212. Eight validation documents hold only a few dozen entities of the rare types, so one document changes macro-F1 by several points. We therefore rely on the leaderboard for ranking systems and on local runs only for diagnosing failure modes. Phase 4 (about 2,000 balanced dev cases) is the exception: Arch 1's best quick-dev F1 (0.752) was close to its leaderboard score (0.7492).

5.3 Ablations and controlled comparisons

Phase 1: baseline versus modified. Table 4 compares each pair. Only Arch 2 is a clean comparison (both ran 762 steps), and there the modified model has no better perplexity (21.27 against 20.35) but scores +0.038 BERTScore. In Arch 1 and Arch 4 the modified model ran 2× and 4× more optimizer steps than the baseline, so some of its perplexity advantage (21.2→10.4 and 22.6→12.0) may come from step count and batch size rather than architecture; we did not rerun with matched steps (a limitation). The Arch 3 baseline and modified scores are 0.8189 and 0.8299, but we have no training logs for the baseline.

Table 4: Phase-1 baseline vs. modified. Loss is on the full validation set (Arch 4: continuation only, so not comparable across families). BS-F1 from the leaderboard.

Arch	Variant	Params (M)	Steps	Min	Val loss	PPL	BS-F1
1	baseline	158.7	762	223	3.053	21.18	0.8299
1	modified	145.0	1525	720	2.342	10.40	0.8449
2	baseline	158.7	762	247	3.013	20.35	0.8018
2	modified	145.0	762	771	3.057	21.27	0.8399
3	baseline	165.8	1525	253	3.031	22.98	0.8189
3	modified	154.3	1525	654	2.830	20.23	0.8299
4	baseline	172.8	1525	141	3.117	22.58	0.8086
4	modified	189.4	6103	416	2.487	12.02	0.8238

* quick 128-sequence validation at step 1400; not comparable with the full-validation figures.

Adaptation methods. Table 5 compares full fine-tuning with parameter-efficient methods (leaderboard scores). In Phase 2, parameter-efficient adaptation beat full fine-tuning for Arch 1 and 2 (LoRMA +0.022 and +0.030). In Phase 3, full fine-tuning was never best; prefix tuning was the worst method for Arch 1 (0.442) but the best for Arch 2 (0.487). LoRMA and LoRA differ by under 0.01 in most cells, which is within the noise of such small test sets.

Table 5: Phase 2 (strict macro-F1) and Phase 3 (macro-F1) leaderboard scores by adaptation method.

Phase	Arch	Full	LoRA	LoRMA	Prefix	Other
2	1	0.5393	0.5530	**0.5610**	–	–
2	2	0.5206	0.5490	**0.5501**	–	–
2	3	–	**0.4911**	0.4885	–	BitFit 0.4742
2	4	–	–	–	–	KD+LoRMA 0.2959
3	1	0.4911	**0.5102**	0.5061	0.4416	–
3	2	0.4551	0.4574	0.4700	**0.4866**	–
3	3	0.4678	**0.4936**	0.4621	n/a	–
3	4 (KD)	**0.3627**	0.2965	0.2419	n/a	–

Distillation. In Phase 3 the 4-layer Arch 4 student distilled from the 8-layer teacher scored 0.330 on local dev, against 0.279 for the same student trained without distillation, and above its own teacher (0.304) with 71% of the parameters and 1.65× the speed. In Phase 2 the same student reached 0.377 with distillation and 0.378 without, so distillation gave no gain there.

Phase 7 retrieval ablation (Arch 1, dev F1@7). The default (mean pooling, four chunks, cosine, InfoNCE, hard negatives) scored 0.265. A pairwise margin loss gave 0.286 (best); in-batch

negatives only gave 0.262 and trained 29% faster; last-token pooling gave 0.251; dot product 0.228; and reading one chunk instead of four 0.220, at a quarter of the training time. A frozen Arch 3 scored 0.229 zero-shot against 0.242 after LoRMA, so training added only 1.3 points.

6 Results

Table 6 gives the leaderboard score of every system we submitted, and Figure 2 shows Phase-1 learning curves. Training curves for the downstream phases are in the notebooks; we show the Phase-1 curves because they carry the central comparison.

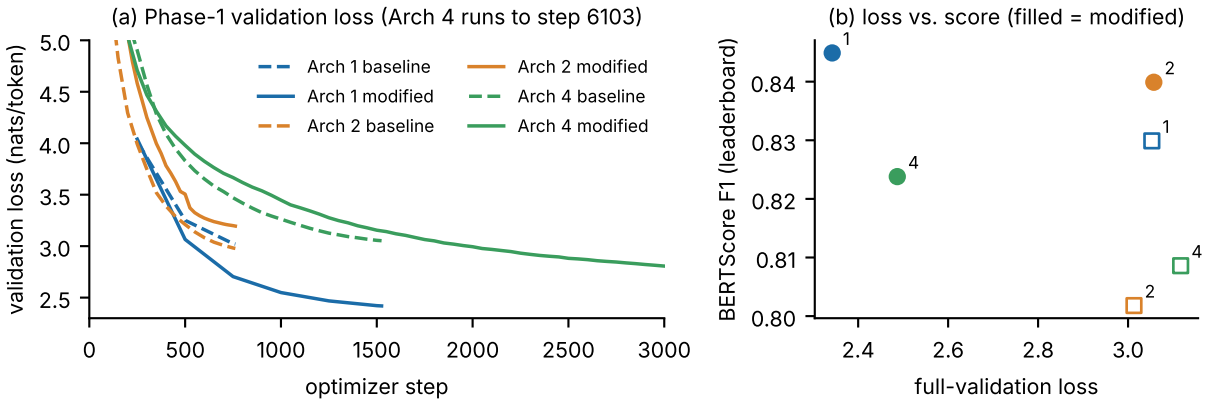


Figure 2: (a) Phase-1 validation loss against step for baseline (dashed) and modified (solid) runs. (b) Full-validation loss against leaderboard BERTScore F1: halving perplexity bought little score, and Arch 2 gained 0.038 without lower loss.

Table 6: Best leaderboard score per phase and architecture, with the adaptation method. Bold = best in row.

Phase	Metric	Arch 1	Arch 2	Arch 3	Arch 4
1	BS-F1	**0.8449**	0.8399	0.8299	0.8238
2	strict F1	**0.5610** LoRMA	0.5501 LoRMA	0.4911 LoRA	0.2959 KD+LoRMA
3	macro-F1	**0.5102** LoRA	0.4866 prefix	0.4936 LoRA	0.3627 KD, full
4	macro-F1	**0.7492** LoRMA	0.6069 LoRA	0.5060 adapter	0.5511 soft prompt
5	macro-F1	**0.3559** LoRMA	0.3483 LoRA	0.3376 LoRA	0.2272 LoRA
6	BS-F1	**0.8437** LoRA	0.8280 LoRMA	0.8377 adapter	0.7956 soft prompt
7	F1@7	**0.2304** LoRA	0.2085 adapter	0.2120 LoRMA	0.0118 soft prompt

Which system won. Arch 1 (decoder) is first in every phase, and Arch 4 is last in six of seven. Our reading is that the ordering follows Phase-1 language-model quality: Arch 1 reached by far the lowest validation perplexity (10.4), and the downstream ranking mostly matches the Phase-1 ordering. Two further factors plausibly add to it, each of which we only partly tested. First, the encoder–decoder's encoder had a 768-token window and was trained to model only the continuation, so its encoder states may be a poor basis for classification; the Phase-7 zero-shot score of 0.023 for Arch 4 against 0.229 for Arch 3 supports this. Second, some downstream runs for Arch 3 and Arch 4 used earlier Phase-1 checkpoints than our final ones (see Section 7), so part of their deficit is under-training rather than architecture. The one phase where the decoder's margin is largest, Phase 4 (0.7492 against 0.607 for the next system), is also the phase in which only Arch 1 beats a TF-IDF baseline (0.695 on local dev).

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phase 1. Perplexity and BERTScore are only loosely related (Figure 2b): Arch 1 halved perplexity for +0.015, and Arch 2 gained 0.038 with equal perplexity. The leaderboard scores of all our systems span only four points (0.82–0.84 for the modified models). **Phase 2.** Training loss falls to near zero (0.003) while validation F1 plateaus after epoch 8 and oscillates (0.547–0.589 for Arch 1), a clear sign of overfitting on 62 documents; BIO-constrained Viterbi decoding gave +0.045 over greedy and the confidence threshold another +0.02. **Phase 3.** The dominant confusions are between opposing sides of the argument: one argument side is labelled as the other (17–29% of the gold sentences for Arch 1–3, and 53% of Respondent arguments for Arch 4), and Ratio of the Decision is confused with Precedent Relied Upon (10–24%). Both pairs are lexically near-identical and differ only in the position in the judgment and the party named. The 3 training examples of Precedent Overruled cannot be learned.

Phase 4. We measured what the model reads (Arch 2, dev macro-F1 at threshold 0.5): the first 1,024 tokens give 0.523, the last 1,024 give 0.573 (0.638 after threshold tuning), head+tail 0.582 and the full case 0.587. Learned attention concentrates at the end of the judgment (1.76× uniform in the last fifth, 0.27× in the middle fifth), which confirms that the outcome sits at the end. Accuracy is flat across document-length quartiles (0.62–0.66), so truncation is not the main error source; the weakness is the classifier itself. The Arch 4 soft-prompt model collapsed to predicting class 0 at the default threshold (macro-F1 0.333; recall for class 1 was zero), and only threshold tuning (0.568) recovered it. Arch 2's calibration error is moderate (ECE 0.073) while Arch 4's is 0.149 [28].

Phase 5. The Arch 4 adapter run produced NaN losses from the first epoch (macro-F1 0.0002). On dev, the other Arch 4 methods reached 0.235 (LoRA), 0.188 (LoRMA) and 0.093 (soft prompt), while Arch 1's LoRMA reached 0.361. Attention analysis for Arch 1 shows that the evidence window of correct predictions lies in the first third (41%) or last third (40%) of a document and rarely in the middle (19%); in the two false-positive examples we inspected, the attention sat on the first window. **Phase 6.** All fine-tuned models score below the lead baseline on ROUGE-L (0.266 against 0.175–0.194; 0.100 for Arch 4), and for Arches 2–4 fine-tuning did not raise ROUGE-1 above the pre-fine-tuning model. Predictions average 160–230 words against 780 in the references, because of our 256-token cap, so recall-oriented overlap is limited. **Phase 7.** For the Arch 3 model, 29 of 118 dev queries have no relevant case in the top seven, the median rank of a relevant case is 34 of 1,023, and 89% of queries are truncated by the four-chunk window. Gold cases per query average 8.2, so recall@7 is bounded.

7.2 What the metrics do not measure

BERTScore and ROUGE-L reward similarity to one hidden reference; neither checks that a cited section, party or holding is correct, and a four-point spread between our best and worst Phase-1 systems is hard to interpret in legal terms. Strict span-F1 gives zero credit to a prediction off by one character, so a model that finds every entity but trims a trailing full stop scores like one that misses it; for a firm doing contract or case review, typed approximate spans are often useful. Macro-F1 in Phase 3 weights the 3-example role equally with the 4,000-example one. Micro-F1@7 caps recall at seven results even when a query has 92 relevant cases. Deployment would need human review and faithfulness checks that none of these metrics provide.

7.3 Limitations and what we would do next

Our largest limitation is that compute (two T4 GPUs and 12-hour sessions) allowed one epoch over 200–400M tokens, so Phase-1 models are far from converged (validation loss still falling at the last step). The baseline-versus-modified comparison is confounded by step count and batch size for Arch 1 and 4, and Arch 3 lacks baseline logs. Local validation sets are tiny, which makes method comparisons of under one point meaningless. We did not run multiple seeds. Next, we would (1) rerun the Phase-1 pairs with identical steps, batch size and tokens, and three seeds; (2) move from the first four chunks to full-document retrieval and add a cross-encoder re-ranker for Phase 7; (3) raise the summarization cap to 800 tokens with chunk-then-merge and compare against an extractive lead-plus-selection hybrid; (4) add a Phase-2 span-level CRF and cross-validation over documents to tighten the local estimate; and (5) evaluate faithfulness of summaries with a statute-and-party consistency check. Because every phase is fine-tuned separately, there is no error propagation, but also no sharing of the structure found in Phases 2–3 (e.g. NER spans as features for Phase 5).

7.4 Surprising findings

Three results contradicted our prior. (i) Halving perplexity (Arch 1) moved BERTScore by only 0.015, while Arch 2 gained 0.038 with a slightly worse perplexity: pre-training loss was a poor guide to the generation metric. (ii) Full fine-tuning was never best in Phases 2 and 3 despite having the most capacity, and the multiplicative LoRMA update matched or beat LoRA in only three of the seven Phase-2/3 comparisons. (iii) In Phase 4 the backbone mattered far more than the adaptation recipe: similar recipes gave 0.7492 with Arch 1 and 0.506 with Arch 3, and a TF-IDF model (0.695 on dev) scored above every architecture except Arch 1.

8 Conclusion

Two design choices moved the metrics. First, the choice of backbone (the decoder-only Arch 1) decided the ranking in all seven phases, with the largest margin on the long-document task. Second, parameter-efficient adaptation (LoRA/LoRMA) was at least as good as full fine-tuning on every small dataset while training 0.4–7% of the parameters. Several choices did not move the metrics much: per-label thresholds in Phase 5, the choice between LoRA and LoRMA, and in retrieval the pooling and negative-sampling options (changes of 0.2–1.3 points), whereas the similarity function and the number of chunks read cost 3.7 and 4.4 points. The main lesson for training a compact legal language model under a parameter and data lock is that the pre-training budget and the quality of the final checkpoint matter more than the cleverness of the attention variant, and that a controlled ablation (equal steps, batch size and tokens) must be planned from the start because late comparisons are confounded.

LLM Usage Declaration

Q1. Claude (Anthropic, Claude Sonnet 5.5).
Q2.

Part	Used LLM?	Purpose
Understanding concepts / papers	Y	explained how LoRMA, GLA and RetNet work
Ideation / brainstorming	Y	brainstormed ablations and long-document strategies
Debugging small ancillary snippets	Y	fixed shape and CUDA out-of-memory errors in notebooks
Plotting / visualization code	Y	Script for the Phase-1 loss figure (Figure 2).
Grammar / writing style of the report	Y	Claude was used to proofread the report, improve grammar, refine sentence structure, and enhance readability.
Report structure / section wording	N	
Designing the model architecture	N	
Hyperparameter search / tuning	N	
Analysis of results / conclusions	N	

Q3. The papers in the bibliography; the course dataset (`Exploration-Lab/CS-781-Capstone`); our tokenizer from the earlier assignment (Kaggle dataset `kratikasum/tokenizer`); the BERTScore and ROUGE implementations. The course starter notebook and evaluation scripts, if used.
Q4. I pre-trained four architecture families (decoder, local/global hybrid, linear-attention hybrid and encoder–decoder), each as a baseline and a modified version, from scratch on 400M tokens of Indian legal text with our BPE tokenizer. I then used the Phase-1 weights as the starting point for six downstream tasks and compared full fine-tuning with LoRA, LoRMA, adapters, prefix and soft prompts, BitFit and distillation. For NER, I found that whitespace splitting recovered only 66% of the gold spans, so I switched to word-plus-punctuation splitting (99.4%) and checked the pipeline with an oracle test whose ceiling is 0.994. One thing that failed was the Phase-5 adapter run for the encoder–decoder, whose loss became NaN in the first epoch (macro-F1 0.0002); I did not isolate the cause, and a likely suspect is fp16 with a high adapter learning rate, which I did not test. A second failure was the Phase-4 soft prompt for the same model, which predicted only class 0 at the default threshold (macro-F1 0.333); tuning the threshold recovered 0.568, so I concluded that its scores were poorly calibrated rather than uninformative. A third was the linear-attention model, whose early checkpoint (step 600, validation loss 3.58) was used in Phase 2 before I retrained it to step 1400 (2.83); I concluded that every downstream result must record the exact checkpoint it used. In Phase 6 every fine-tuned model scored below the lead baseline on ROUGE-L (0.266), which I link to my 256-token generation cap against 780-word reference summaries. The decoder was best in all seven phases, and parameter-efficient methods matched or beat full fine-tuning in Phases 2 and 3. Comparing baselines with modified models was confounded by different step counts, so I treat those gains as indicative only.

References

[1] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[2] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[4] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of ACL*, 2025.

[5] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *Proceedings of ICML*, 2019.

[6] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of ACL*, 2021.

[7] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of EMNLP*, 2021.

[8] E. Ben Zaken, Y. Goldberg, and S. Ravfogel, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of ACL*, 2022.

[9] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[10] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of ACL*, 2016.

[11] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems*, 2019.

[12] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[13] H. Touvron *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[14] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, 2024.

[15] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of EMNLP*, 2023.

[16] A. Henry, P. R. Dachapally, S. Pawar, and Y. Chen, "Query-key normalization for transformers," in *Findings of EMNLP*, 2020.

[17] Z. Zhou, T. Wu, Z. Jiang, and Z. Lan, "Value residual learning for alleviating attention concentration in transformers," *arXiv preprint arXiv:2410.17897*, 2024.

[18] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve, "Better & faster large language models via multi-token prediction," in *Proceedings of ICML*, 2024.

[19] A. Q. Jiang *et al.*, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.

[20] A. Kazemnejad, I. Padhi, K. Natesan Ramamurthy, P. Das, and S. Reddy, "The impact of positional encoding on length generalization in transformers," in *Advances in Neural Information Processing Systems*, 2023.

[21] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei, "Retentive network: A successor to transformer for large language models," *arXiv preprint arXiv:2307.08621*, 2023.

[22] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim, "Gated linear attention transformers with hardware-efficient training," in *Proceedings of ICML*, 2024.

[23] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[24] S. Kumar and W. Byrne, "Minimum Bayes-risk decoding for statistical machine translation," in *Proceedings of HLT-NAACL*, 2004.

[25] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proceedings of EMNLP*, 2014.

[26] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, 2023.

[27] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[28] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of ICML*, 2017.